Post-Order Workflows — Implementation Plan

Field	Value
Document ID	MNM-IMPL-POST-ORDER-001
Version	1.0
Date	8 September 2026
Project	Mini's n More — commerce platform
Baseline	Customer Cancel · Customer Return · Exchange request sequence diagrams (30 Aug 2026)
Related	admin-independent-commerce-operations-requirements.md · order-status-lifecycle.md · order-return-json-contract.md

---

1. Purpose

This document is the **implementation plan** for three post-order workflows supplied as sequence diagrams:

1. **Customer cancel request** — pre-dispatch order cancellation with admin decision and refund
2. **Customer return request** — post-delivery return, reverse logistics, inspection and refund
3. **Exchange request** — post-delivery exchange, reverse logistics, disposition, price adjustment and replacement delivery

The diagrams define the **target operating model**. This plan describes **what will be built** — customer surfaces, admin workspace, system rules, carrier integration, payments and notifications — so that each diagram step is supported end-to-end in the Mini's n More platform.

Mini's n More admin remains the **system of record**. Zoho Books, Ledgerly and XpressBees are integrated asynchronously and must not block local operations.

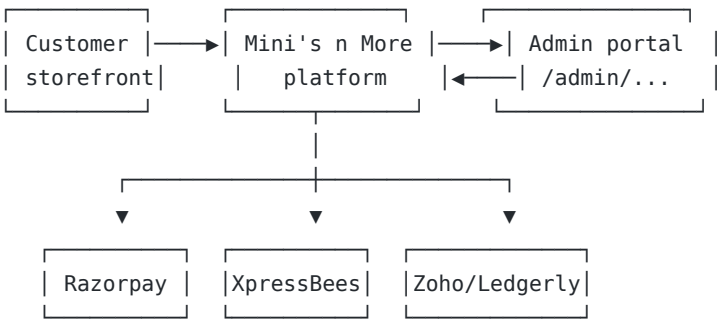
---

2. Scope

Area	Implementation deliverable
Cancellation	Customer request, admin approve/reject, refund, stock release, notifications
Return	Eligibility, admin workflow, XpressBees reverse pickup (with manual fallback), inspection, disposition, line-level refund
Exchange	Variant selection, reservation, reverse pickup, disposition, price difference, replacement shipment, case closure
Logistics	XpressBees reverse pickup and outbound AWB; webhooks; pickup-failure handling
Payments	Razorpay refunds; COD manual refund path; refund retry and idempotency
Inventory	Reservation, reverse transit, inspection-driven restock only
Accounting	Zoho / Ledgerly sync after local confirmation (non-blocking)
Notifications	Email and SMS at each customer-visible transition

---

3. Actors and integrations



Actor / system	Responsibility
Customer	Submit cancel / return / exchange; choose replacement variant; track case status; withdraw request before processing
Admin	Review requests; book or confirm logistics; inspect goods; set disposition; approve refunds; close cases
System	Eligibility, state machine, inventory, payments, idempotent webhooks, audit trail
XpressBees	Reverse pickup scheduling, collection updates, outbound AWB and delivery tracking
Razorpay	Online payment refunds
Zoho / Ledgerly	Credit notes and order cancellation sync

---

4. Shared platform design

These capabilities will be implemented once and reused across all three workflows.

4.1 Case model

Every cancellation, return and exchange will be a **case** with:

- unique reference (`CAN-…` , `RET-…` , or shared return table with `request_type`)
- linked order and line items / quantities
- reason, admin notes, rejection reason
- status history and actor audit
- optional carrier fields (provider id, AWB, tracking URL, pickup window)
- refund record(s) where applicable

4.2 State machine rules

- Invalid transitions will be rejected (HTTP 400).
- Terminal states: `closed` , `rejected` , `cancelled` , `refunded` , `completed` .
- Concurrent admin actions and duplicate webhooks will be idempotent.
- Customer-visible labels will be distinct from internal admin states.

4.3 Inventory rules

Event	Stock behaviour
Order placed	Reserve sellable quantity
Cancel approved	Release reservation

Return/exchange approved	**No** sellable restock yet
Item received + inspected + disposition `restockable`	Increase sellable stock
Exchange approved	**Reserve** replacement variant
Exchange rejected / converted to refund	Release replacement reservation once
Disposition `discard` / `return_to_vendor`	No sellable increase

4.4 Notifications

Trigger	Customer	Admin
Request submitted	Email + SMS	Queue entry
Approved / rejected	Email + SMS	—
Pickup scheduled / failed	Email + SMS	Email / webhook alert
Refund initiated / completed / failed	Email + SMS	Dashboard flag
Replacement shipped / delivered	Email + SMS with tracking link	—
Case closed	Email + SMS	—

4.5 Admin workspace

Surface	Path	Purpose
Returns & exchanges queue	`/admin/returns`	Filter by status and type; next-action buttons
Order detail	`/admin/orders`	Cancel case review; link to open post-order cases
Refund reconciliation	Admin finance view	Retry failed refunds; manual COD settlement

4.6 Customer workspace

Surface	Path	Purpose
Order detail	`/account/orders/{orderNumber}`	Cancel, return, per-line exchange
My returns	`/account/returns`	All cases and statuses

---

5. Workflow 1 — Customer cancel request

5.1 Target sequence (diagram)

```
Customer → Admin: Place cancel request
Admin: If valid reason → Accept
  → Approve → Initiate refund → Customer
  OR
  → Reject → Cancel rejected → Customer
```

5.2 Implementation flow

```
Customer submits cancellation request + reason
  → System: local eligibility check (not shipped / not delivered)
  → Status: CANCELLATION_REQUESTED (pending admin review)
      → Admin APPROVES
          → Stop conflicting fulfillment
          → Release reserved stock
          → Initiate Razorpay refund if paid online
          → Record COD manual refund path if collected COD
          → Sync Zoho / Ledgerly (async)
          → Notify customer: cancelled / refund processing / refunded
          → CLOSED
      → Admin REJECTS (reason required)
          → Order continues fulfillment
          → Notify customer: cancellation rejected
          → CLOSED
```

Configurable **auto-approve** for safe pre-fulfillment requests may be added; it will produce the same audit trail and side effects as manual approval.

5.3 Features to implement

ID	Feature	Description
CAN-01	Customer cancel request	Whole-order request from order detail; reason required (min length)
CAN-02	Eligibility engine	Local rules: not cancelled, not shipped/delivered/fulfilled; no Zoho dependency
CAN-03	Admin review queue	List pending cancellations; approve / reject with reason
CAN-04	Fulfillment hold	Pending cancellation blocks new shipment booking unless admin overrides
CAN-05	Refund on approval	Razorpay refund for online paid orders; idempotent retry on failure
CAN-06	COD handling	No gateway call for unpaid COD; manual refund workflow for collected COD
CAN-07	Carrier cancel attempt	If label exists but not handed over, async XpressBees cancel + manual flag
CAN-08	Stock release	Atomic release of reservations for cancelled quantity only
CAN-09	Customer status labels	`cancellation requested`, `cancelled — refund pending`, `cancelled — refunded`, `cancellation rejected`
CAN-10	Notifications	Email on submit, approve, reject and refund outcome

5.4 APIs (planned)

Method	Endpoint	Actor
POST	`/api/v1/orders/{orderNumber}/cancel-request`	Customer — submit request

GET	`/api/v1/orders/{orderNumber}/cancel-request`	Customer — view pending case
GET	`/api/v1/admin/cancellation-requests`	Admin — queue
POST	`/api/v1/admin/cancellation-requests/{id}/approve`	Admin
POST	`/api/v1/admin/cancellation-requests/{id}/reject`	Admin

Note: instant cancel endpoint may remain for auto-approved eligible orders; admin-reviewed path is the primary diagram flow.

5.5 Acceptance criteria

- Customer can request cancellation with reason before dispatch.
- Admin can approve or reject; rejection preserves fulfillment.
- Paid online approval triggers exactly one Razorpay refund.
- Unpaid COD approval never calls the payment gateway.
- Customer receives distinct notifications for approve, reject and refund.
- Shipment cannot proceed while cancellation is pending without audited override.

---

6. Workflow 2 — Customer return request

6.1 Target sequence (diagram)

```
Customer → Admin: Place return request
Admin: Approve
Admin → ExpressBees: Request for pickup
ExpressBees: Inform customer, collect item, return to Admin
ExpressBees → Admin: Return item
Admin: Approve → Initiate refund → Customer
   OR Reject → Return closed → Customer

Alternate – pickup failure:
ExpressBees → Admin: Inform (email / call / webhook)
  Reasons: customer unavailable / no product / changed mind
Admin → Customer: reschedule or close case
```

6.2 Implementation flow

```
REQUESTED
  → Admin APPROVED
  → PICKUP_REQUESTED → book XpressBees reverse pickup (or manual / self-ship)
  → PICKUP_SCHEDULED → customer notified
  → PICKED_UP → IN_TRANSIT
  → RECEIVED → warehouse receipt recorded
  → INSPECTED → condition + line qty captured
       → ACCEPTED → disposition per unit (restockable / quarantine / discard / vendor)
            → REFUND_PENDING → calculate line-level refund breakdown
            → REFUNDED → Razorpay + accounting sync → CLOSED
       → REJECTED_AFTER_INSPECTION → return to customer / exceptional refund → CLOSED
  → PICKUP_FAILED → RESCHEDULED or CANCELLED (standardized reason)
  → REJECTED (from REQUESTED) → CLOSED
  → CANCELLED_BY_CUSTOMER (before operational processing) → CLOSED
```

6.3 Features to implement

ID	Feature	Description
RET-01	Return request	Customer selects order lines and quantities; reason required
RET-02	Eligibility	Delivered + paid + within return window (configurable days) + no active case
RET-03	Admin approve / reject	Review queue; rejection reason shown to customer
RET-04	XpressBees reverse pickup	Book from shipping address; store AWB, tracking URL, pickup window
RET-05	Manual logistics fallback	Admin selects manual carrier or customer self-ship with tracking
RET-06	Pickup failure handling	Webhook / poll maps failure reason; admin reschedule or close
RET-07	Warehouse receipt	Partial or full receipt; receiver, timestamp, optional photos
RET-08	Inspection	Accepted / partially accepted / rejected with line-level qty
RET-09	Disposition	`restockable`, `quarantine`, `repair`, `discard`, `return_to_vendor`
RET-10	Restock rule	Sellable stock increases **only** after `restockable` disposition
RET-11	Line-level refund	Prorated amount from order snapshot; admin sees breakdown before confirm
RET-12	Refund execution	Razorpay for online; manual UPI/bank for COD with audit fields
RET-13	Refund retry	`failed` state with admin retry; no duplicate money movement
RET-14	Notifications	Pickup scheduled, failed, collected, refund processed

6.4 Status model

Status	Customer label (example)	Next admin action
`requested`	Return requested	Approve / Reject
`approved`	Return approved	Book pickup
`pickup_requested`	Pickup being arranged	Confirm scheduled
`pickup_scheduled`	Pickup scheduled	Mark picked up
`picked_up`	Item collected	Mark received
`received`	Item at warehouse	Inspect

`inspected`	Under review	Accept / Reject
`accepted`	Return accepted	Issue refund
`refund_pending`	Refund processing	Confirm gateway
`refunded`	Refunded	—
`pickup_failed`	Pickup issue	Reschedule / Close
`rejected`	Return declined	—
`cancelled`	Withdrawn	—

6.5 APIs (planned)

Method	Endpoint	Actor
POST	`/api/v1/order-returns`	Customer — create (with line items)
GET	`/api/v1/order-returns/eligibility/{orderNumber}`	Customer
POST	`/api/v1/order-returns/{returnNumber}/cancel`	Customer — withdraw
POST	`/api/v1/admin/order-returns/{returnNumber}/approve`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/reject`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/book-pickup`	Admin — XpressBees
POST	`/api/v1/admin/order-returns/{returnNumber}/pickup-failed`	Admin / webhook
POST	`/api/v1/admin/order-returns/{returnNumber}/mark-received`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/inspect`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/accept`	Admin + disposition
POST	`/api/v1/admin/order-returns/{returnNumber}/refund`	Admin
POST	`/webhooks/xpressbees/returns`	Carrier

6.6 XpressBees reverse pickup (planned)

On admin **Book pickup**:

1. Send pickup address, contact, SKU/qty, case reference to XpressBees.
2. Persist provider request id, reverse AWB, tracking URL, scheduled window.
3. Notify customer of pickup schedule.

On carrier webhook / poll:

Carrier event	Local transition
Out for pickup	`pickup_scheduled`
Picked up	`picked_up`
In transit to hub	`in_transit`
Delivered to warehouse	`received` (or await admin confirm)

Failed — customer NA	`pickup_failed` + reason
Failed — no product	`pickup_failed` + reason
Failed — customer cancelled	`pickup_failed` + reason

Admin will be notified by **webhook**, email and dashboard alert on failure.

6.7 Acceptance criteria

- Customer can return one line from a multi-item order without refunding the full order.
- Approval does not restock or refund automatically.
- XpressBees booking stores AWB and tracking; manual path works when API unavailable.
- Pickup failure allows reschedule or case closure with customer notification.
- Refund breakdown visible to admin before confirmation.
- Accounting sync failure does not revert local `refunded` status.

---

7. Workflow 3 — Exchange request

7.1 Target sequence (diagram)

```
Customer → System: Place exchange request
System: Accept if condition OK, inventory available, valid reason
System → Admin: Allow exchange
Admin: Approve
Admin → ExpressBees: Initiate pickup (reverse)
ExpressBees: Schedule pickup, inform customer, collect → mark picked
ExpressBees → Admin: Handover
Admin: Accept return
  - Pack replacement / initiate outbound delivery
  - Decide returned item: restock / dispose / return to vendor
  - Price difference: contact customer and settle
Admin → ExpressBees: Initiate replacement delivery
ExpressBees: AWB, out for delivery, tracking link, delivered
ExpressBees → Admin: Inform delivered
Admin → Customer: Close exchange

Reject: Admin reject → inform customer
Pickup failure: ExpressBees → Admin → exchange cancelled
```

7.2 Implementation flow

```
REQUESTED
  → System validates: delivered, paid, variant product, sellable replacement, valid reason
  → Admin APPROVED + reserve replacement stock atomically
  → reverse pickup lifecycle (same as return §6.2)
  → RECEIVED → INSPECTED
       → ACCEPTED
           → settle price difference (collect / refund / waive with audit)
           → ITEM_ALLOCATED → REPACKING → READY_TO_SHIP
           → book XpressBees outbound OR manual AWB
           → SHIPPED → DELIVERED
           → COMPLETED → notify customer → CLOSED
       → CONVERTED_TO_REFUND → item refund lifecycle → CLOSED
```

```
      → REJECTED_AFTER_INSPECTION → release reservation → resolution → CLOSED
  → REJECTED (from REQUESTED) → release reservation → CLOSED
  → PICKUP_FAILED → reschedule OR exchange cancelled → CLOSED
```

7.3 Features to implement

ID	Feature	Description
EXC-01	Exchange request	Per order line; customer picks replacement size/colour from sellable variants
EXC-02	System validation	Reason, eligibility, replacement availability check at submit
EXC-03	Replacement reservation	Atomic reserve on admin approval
EXC-04	Reverse pickup	Reuse return logistics (§6.6)
EXC-05	Disposition	Same disposition model as return for returned unit
EXC-06	Price difference	Snapshot policy; collect top-up or refund delta before dispatch
EXC-07	Replacement fulfilment	Allocate → repack → ship → deliver workflow in admin
EXC-08	Outbound XpressBees	AWB, label, tracking URL; webhook maps to shipped/delivered
EXC-09	Manual outbound fallback	Admin enters carrier + AWB when API unavailable
EXC-10	Convert to refund	Admin converts accepted exchange to refund without duplicate case
EXC-11	Close exchange	Case completes only after confirmed delivery or authorized manual close
EXC-12	Pickup failure	Same failure reasons as return; admin can cancel exchange

7.4 Price difference handling (planned)

Scenario	Implementation
Replacement price **higher**	Admin records payment collection (link / UPI / waiver); block dispatch until confirmed
Replacement price **lower**	Partial refund through refund subsystem before or on dispatch
Same price	No adjustment; audit note only
Policy	Version snapshotted on case; shown to customer at request time

7.5 Disposition after exchange return (planned)

When admin accepts the returned item:

Disposition	Action
`restockable`	Add to sellable inventory after inspection
`quarantine` / `repair`	Non-sellable bucket
`discard`	Write-off with audit
`return_to_vendor`	Vendor adjustment ledger

Original item will **not** re-enter sellable stock at approval or pickup — only after restockable disposition.

7.6 Status model (exchange-specific tail)

Shared through `accepted` with return; then:

Status	Admin action
`accepted`	Settle price difference
`item_allocated`	Mark repacking
`repacking`	Mark ready to ship
`ready_to_ship`	Book outbound / enter AWB
`shipped`	Mark delivered (or wait for webhook)
`delivered`	Complete exchange
`completed`	—

7.7 APIs (planned)

Method	Endpoint	Actor
POST	`/api/v1/order-returns/exchange`	Customer
GET	`/api/v1/order-returns/exchange-availability/{orderItemId}`	Customer
POST	`/api/v1/admin/order-returns/{returnNumber}/approve`	Admin — includes reservation
POST	`/api/v1/admin/order-returns/{returnNumber}/settle-price-difference`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/allocate-replacement`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/book-outbound`	Admin — XpressBees
POST	`/api/v1/admin/order-returns/{returnNumber}/convert-to-refund`	Admin
POST	`/api/v1/admin/order-returns/{returnNumber}/complete`	Admin

7.8 Acceptance criteria

- Customer can request exchange for a variant line with chosen replacement.
- Approval reserves replacement; two cases cannot reserve the last unit.
- Returned unit restocked only via restockable disposition.
- Price difference settled and audited before replacement ships.
- Outbound AWB and tracking sent to customer.
- Exchange closes only after delivery confirmation.
- Pickup failure can cancel or reschedule the exchange with notification.

---

8. Delivery — Phase 2

All post-order workflow deliverables below are planned for **Phase 2**. Together they implement the full diagram-aligned target: core cases, logistics automation, inventory/disposition and finance depth.

8.1 Core cases and admin workspace

Deliverable	Workflows
Return / exchange case entity and admin queue	Return, Exchange
Customer request, eligibility, withdraw	Return, Exchange

Admin approve / reject / inspect / accept	Return, Exchange
Razorpay refund + retry	Return, Exchange, Cancel
Email / SMS notifications	All
Storefront and admin UI next-action buttons	All
Cancellation with admin review queue	Cancel
Line-level return selection and refund calculation	Return

8.2 XpressBees logistics automation

Deliverable	Workflows
Reverse pickup booking API	Return, Exchange
Outbound replacement AWB booking	Exchange
Webhook ingestion + idempotent status mapping	Return, Exchange
Pickup failure reasons and reschedule	Return, Exchange
Manual carrier / self-ship fallback	Return, Exchange

8.3 Inventory, disposition and finance depth

Deliverable	Workflows
Disposition model and restock rules	Return, Exchange
Exchange price difference settlement	Exchange
Partial refund payment status on orders	Return, Exchange
COD manual refund audit	All
Carrier cancel on pending order cancellation	Cancel
SLA / queue ageing on admin dashboard	All

---

9. Test and sign-off plan

Each workflow will be verified against the sequence diagram before release.

9.1 Cancellation

ID	Scenario	Expected outcome
CAN-01	Request before ship	Case `cancellation requested`; admin notified
CAN-02	Admin approve paid order	Refund initiated; stock released; customer notified
CAN-03	Admin reject	Order continues; customer sees rejection reason
CAN-04	Request after ship	Blocked; customer directed to return if delivered

9.2 Return

ID	Scenario	Expected outcome
RET-01	End-to-end with XpressBees pickup	Pickup → receive → inspect → refund
RET-02	Pickup failure	Admin alerted; reschedule or close
RET-03	Partial line return	Refund matches line allocation only
RET-04	Reject after inspection	Case closed; disposition resolution recorded

9.3 Exchange

ID	Scenario	Expected outcome
EXC-01	End-to-end with replacement delivery	Reserve → pickup → ship → `completed`
EXC-02	Higher price replacement	Difference collected before dispatch
EXC-03	Convert to refund	Single case; item-level refund
EXC-04	Pickup failure	Exchange cancelled or rescheduled

9.4 Sign-off

Workflow	Test IDs	Pass	Client	Date	Signature
Cancellation	CAN-01 – 04	☐			
Return	RET-01 – 04	☐			
Exchange	EXC-01 – 04	☐			

---

10. Technical reference (implementation anchors)

Component	Location
Return / exchange service	`backend/.../service/OrderReturnService.java`
Order cancellation	`backend/.../InternalAccountOrderResource.java`
Return APIs	`backend/.../InternalOrderReturnResource.java`, `InternalAdminOrderReturnResource.java`
XpressBees client	`backend/.../client/XpressBeesClient.java`
Admin returns UI	`src/routes/admin.returns.tsx`
Customer order UI	`src/routes/account.orders.$orderNumber.tsx`
Status lifecycle doc	`docs/order-status-lifecycle.md`
JSON contract	`docs/order-return-json-contract.md`

---

11. Diagram reference

The implementation plan is traced to these supplied sequence diagrams:

1. **Customer cancel request** — customer request, admin approve/reject, refund
2. **Customer return request** — approve, ExpressBees pickup, receive, approve/refund or reject, pickup failure path

3. **Exchange request** — system validation, admin approve, reverse pickup, disposition, price difference, replacement delivery via ExpressBees, close exchange

Every message and decision node in those diagrams maps to a feature, status transition or integration described in Sections 5–7.

End of document MNM-IMPL-POST-ORDER-001 v1.0